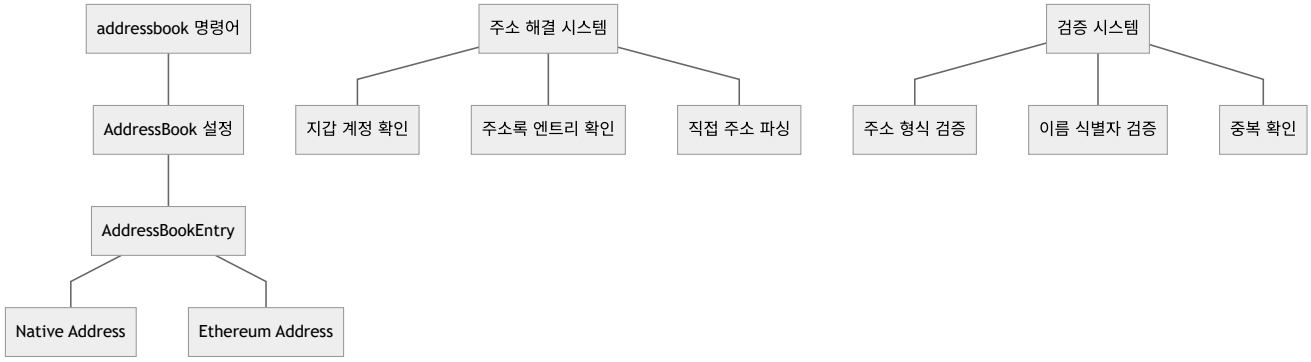
주소록 관리 기능 분석 문서

개요

Oasis CLI의 주소록(Address Book) 관리 기능은 사용자가 자주 사용하는 주소들을 이름으로 저장하고 관리할 수 있는 시스템입니다. 이 기능을 통해 복잡한 주소 문자열 대신 의미 있는 이름을 사용하여 트랜잭션을 수행할 수 있습니다.

주소록 시스템 아키텍처

1. 구성 요소 구조



2. 데이터 모델

AddressBook 구조체

```
type AddressBook struct {
```

```
All  map[string]*AddressBookEntry `mapstructure:",remain"`
}
```

AddressBookEntry 구조체

```
type AddressBookEntry struct {
Description  string  `mapstructure:"description"`
Address  string  `mapstructure:"address"`
EthAddress  string  `mapstructure:"eth_address,omitempty"`
}
```

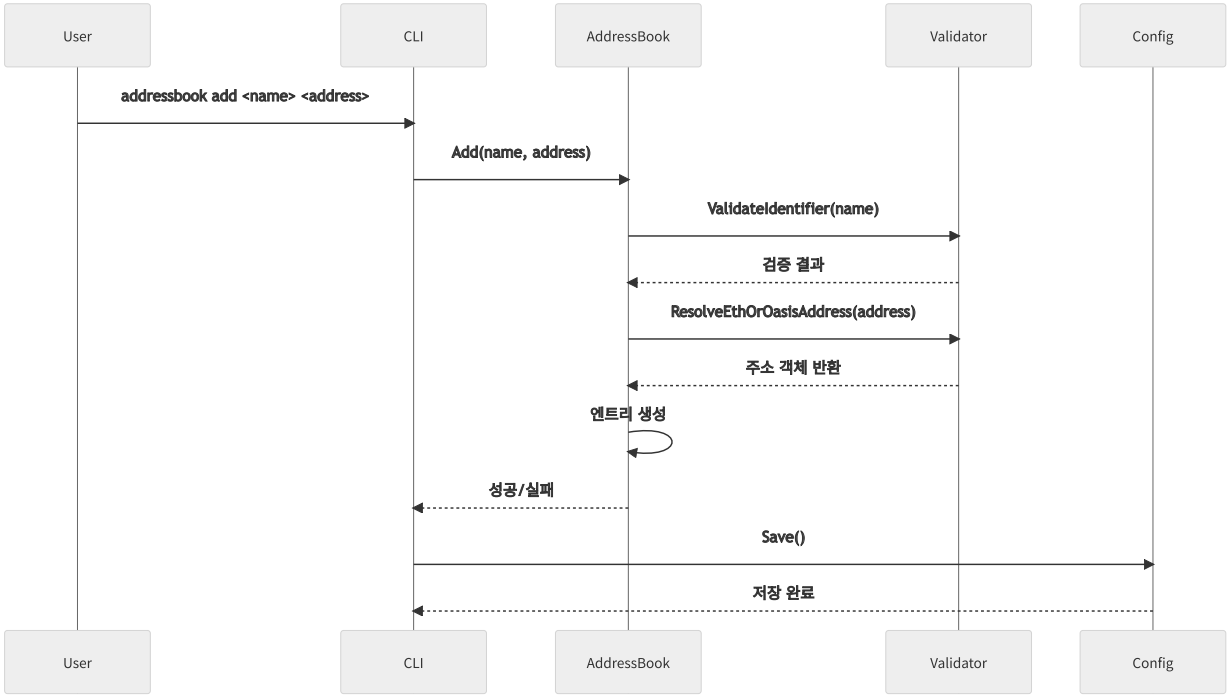
주요 기능 구현 분석

1. 주소록 엔트리 추가 (Add)

구현 위치

- 명령어: `cmd/addressbook.go - abAddCmd`
- 로직: `config/addressbook.go - Add()` 메서드

처리 과정



핵심 구현 코드

```
func (ab *AddressBook) Add(name  string, address  string) error {
// 1. 중복 확인
if  _, exists := ab.All[name];  exists {
return  fmt.Errorf("address named '%s' already exists in the address book", name
}
```

```go
// 2. 이름 검증
if err := config.ValidateIdentifier(name); err != nil {
return fmt.Errorf("malformed address name '%s': %w", name, err)
}

// 3. 주소 해결 및 검증
nativeAddr, ethAddr, err := helpers.ResolveEthOrOasisAddress(address)
if err != nil {
return err
}
if nativeAddr == nil {
return fmt.Errorf("cannot determine address format")
}

// 4. 엔트리 생성
nativeAddrStr, err := nativeAddr.MarshalText()
if err != nil {
return fmt.Errorf("failed to marshal address: %w", err)
}

abEntry := AddressBookEntry{
Address: string(nativeAddrStr),
```

```
}

if ethAddr != nil {
abEntry.EthAddress = ethAddr.Hex()
}

// 5. 저장
if ab.All == nil {
ab.All = make(map[string]*AddressBookEntry)
}
ab.All[name] = &abEntry

return nil
}
```

2. 주소록 목록 조회 (List)

구현 특징

- 이름순 정렬 출력
- 테이블 형태로 표시
- Ethereum 주소가 있는 경우 우선 표시

핵심 구현

```go
func listAddressBook() {
cfg := config.Global()
table := table.New()
table.SetHeader([]string{"Name", "Address"})

var output [][]string
for name, acc := range cfg.AddressBook.All {
addrStr := acc.Address
// Ethereum 주소가 있으면 우선 표시
if ethAddr := acc.GetEthAddress(); ethAddr != nil {
addrStr = ethAddr.Hex()
}
output = append(output, []string{name, addrStr})
}

// 이름순 정렬
sort.Slice(output, func(i, j int) bool {
return output[i][0] < output[j][0]
})

table.AppendBulk(output)
table.Render()
```

```
}
```

3. 주소록 엔트리 조회 (Show)

출력 정보

- 엔트리 이름
- Ethereum 주소 (있는 경우)
- Native Oasis 주소

구현 코드

```
func showAddressBookEntry(name string) {
abEntry, ok := config.Global().AddressBook.All[name]
if !ok {
return fmt.Errorf("address named '%s' does not exist in the address book", name
}

fmt.Printf("Name: %s\n", name)
if abEntry.GetEthAddress() != nil {
fmt.Printf("Ethereum address: %s\n", abEntry.GetEthAddress().Hex())
}
fmt.Printf("Native address: %s\n", abEntry.GetAddress())
}
```

4. 주소록 엔트리 삭제 (Remove)

처리 과정

1. 엔트리 존재 확인
2. 이름 검증
3. 맵에서 삭제
4. 설정 파일 저장

구현 코드

```
func (ab *AddressBook) Remove(name  string) error {
if  _, exists := ab.All[name]; !exists {
return  fmt.Errorf("address named '%s' does not exist in the address book", name
}

if  err := config.ValidateIdentifier(name);  err != nil {
return  fmt.Errorf("malformed address name '%s': %w", name, err)
}

delete(ab.All, name)
return  nil
}
```

5. 주소록 엔트리 이름 변경 (Rename)

처리 과정

1. 기존 이름 존재 확인
2. 새 이름 중복 확인
3. 두 이름 모두 검증
4. 엔트리 이동 및 기존 엔트리 삭제

구현 코드

```
func (ab *AddressBook) Rename(oldName, newName  string) error {
cfg, exists := ab.All[oldName]
if !exists {
return  fmt.Errorf("address named '%s' does not exist in the address book", oldN
}

if  _, exists = ab.All[newName];  exists {
return  fmt.Errorf("address named '%s' already exists in the address book", newN
}

if  err := config.ValidateIdentifier(oldName);  err != nil {
return  fmt.Errorf("malformed old address name '%s': %w", oldName, err)
}
if  err := config.ValidateIdentifier(newName);  err != nil {
return  fmt.Errorf("malformed new address name '%s': %w", newName, err)
}
```

```
ab.All[newName] = cfg

delete(ab.All, oldName)

return  nil

}
```

주소 검증 및 형식 변환 과정

1. 주소 해결 시스템

주소록 시스템은 `helpers.ResolveEthOrOasisAddress()` 함수를 사용하여 다양한 주소 형식을 처리합니다.

지원되는 주소 형식

1. **Native Oasis 주소**: `oasis1qrtrpg56l6y2cfudwtgfuxmq5e5cyhffcsfpdqvw`
2. **Ethereum 주소**: `0xBe8B38ED9b0794e7ab9EbEfC1e710b4F4EC6F6C1`

2. 주소 검증 과정

```
flowchart  TD

A[주소 입력]  -->  B{주소 형식 확인}

B -->|Ethereum|  C[Ethereum 주소 검증]

B -->|Oasis|  D[Oasis 주소 검증]

C -->  E[Native 주소로 변환]

D -->  F[Native 주소 확인]

E -->  G[두 주소 모두 저장]

F -->  H[Native 주소만 저장]
```

```
G --> I[검증 완료]
H --> I
```

검증 구현

```go
func (a *AddressBookEntry) Validate() error {
// 1. 기본 주소 검증
_, _, err := helpers.ResolveEthOrOasisAddress(a.Address)
if err != nil {
return fmt.Errorf("malformed address '%s': %w", a.Address, err)
}

// 2. Ethereum 주소 검증 (있는 경우)
if a.EthAddress != "" {
nativeAddr, _, err := helpers.ResolveEthOrOasisAddress(a.EthAddress)
if err != nil {
return fmt.Errorf("malformed address '%s': %w", a.EthAddress, err)
}
if nativeAddr == nil {
return fmt.Errorf("eth address '%s' was not recognized as valid eth address", a
}
// 3. 주소 일치성 확인
if nativeAddr.String() != a.Address {
```

```
return fmt.Errorf("eth address '%s' (converted to '%s') mismatches stored addre
a.EthAddress, nativeAddr.String(), a.Address)
}
}

return nil
}
```

3. 주소 객체 반환 메서드

Native 주소 반환

```
func (a *AddressBookEntry) GetAddress() types.Address {
var address types.Address
if err := address.UnmarshalText([]byte(a.Address)); err != nil {
panic(err)
}
return address
}
```

Ethereum 주소 반환

```
func (a *AddressBookEntry) GetEthAddress() *ethCommon.Address {
if a.EthAddress != "" {
```

```
_, ethAddr, err := helpers.ResolveEthOrOasisAddress(a.EthAddress)

cobra.CheckErr(err)

return ethAddr

}

return nil

}
```

지갑 시스템과의 연동 방식

1. 주소 해결 우선순위

CLI에서 주소를 해결할 때 다음 순서로 확인합니다:

```
flowchart TD

A[주소 문자열 입력] --> B{지갑 계정 이름?}

B -->|Yes| C[지갑 계정 반환]

B -->|No| D{주소록 엔트리 이름?}

D -->|Yes| E[주소록 엔트리 반환]

D -->|No| F[직접 주소 파싱]

F --> G[주소 객체 반환]
```

구현 코드

```
func ResolveLocalAddress(net *configSdk.Network, address string) (*types.Addre

// 1. 지갑 계정 확인
```

```
if acc, ok := config.Global().Wallet.All[address]; ok {
addr := acc.GetAddress()
return &addr, nil, nil
}

// 2. 주소록 엔트리 확인
if entry, ok := config.Global().AddressBook.All[address]; ok {
addr := entry.GetAddress()
return &addr, entry.GetEthAddress(), nil
}

// 3. 직접 주소 해결
return ResolveAddress(net, address)
}
```

2. 계정 이름 생성 시스템

주소록과 지갑의 모든 엔트리를 통합하여 주소-이름 매핑을 생성합니다:

```
func GenAccountNames() types.AccountNames {
an := types.AccountNames{}
// 지갑 계정 추가
for name, acc := range config.Global().Wallet.All {
```

```
an[acc.GetAddress().String()] = name
}

// 주소록 엔트리 추가
for name, acc := range config.Global().AddressBook.All {
an[acc.GetAddress().String()] = name
}

// 테스트 계정 추가
for name, acc := range testing.TestAccounts {
an[acc.Address.String()] = fmt.Sprintf("test:%s", name)
}

return an
}
```

3. 이름 충돌 방지

지갑 계정 생성 시 주소록 확인

```
// 지갑 계정 생성 시
if _, exists := cfg.AddressBook.All[name]; exists {
return fmt.Errorf("address named '%s' already exists in the address book", name
```

```
}
```

주소록 엔트리 추가 시 지갑 확인

```
// 주소록 엔트리 추가 시
if _, exists := cfg.Wallet.All[name]; exists {
return fmt.Errorf("account '%s' already exists in the wallet", name)
}
```

설정 파일 저장 및 로딩

1. TOML 형식 저장

주소록 데이터는 CLI 설정 파일의 `address_book` 섹션에 저장됩니다:

```
[address_book]

[address_book.mike]
address = "oasis1qrtrpg56l6y2cfudwtgfuxmq5e5cyhffcsfpdqvw"

[address_book.meghan]
address = "oasis1qrjzcve7y6qp3nqs3n7ghavw68vkdh3epcv64ega"
eth_address = "0xBe8B38ED9b0794e7ab9EbEfC1e710b4F4EC6F6C1"
```

2. 설정 검증

시스템 시작 시 모든 주소록 엔트리의 유효성을 검증합니다:

```
func (ab *AddressBook) Validate() error {
for name, a := range ab.All {
if err := config.ValidateIdentifier(name); err != nil {
return fmt.Errorf("malformed address name '%s': %w", name, err)
}

if err := a.Validate(); err != nil {
return fmt.Errorf("address '%s': %w", name, err)
}
}
return nil
}
```

사용 예시 및 시나리오

1. 기본 사용 시나리오

Oasis 네이티브 주소 추가

```
$ oasis addressbook add mike oasis1qrtrpg56l6y2cfudwtgfuxmq5e5cyhffcsfpdqvw
```

Ethereum 주소 추가

```
$ oasis addressbook add meghan 0xBe8B38ED9b0794e7ab9EbEfC1e710b4F4EC6F6C1
```

주소록 목록 조회

```
$ oasis addressbook list
NAME ADDRESS
meghan 0xBe8B38ED9b0794e7ab9EbEfC1e710b4F4EC6F6C1
mike oasis1qrtrpg56l6y2cfudwtgfuxmq5e5cyhffcsfpdqvw
```

특정 엔트리 상세 조회

```
$ oasis addressbook show meghan
Name: meghan
Ethereum address: 0xBe8B38ED9b0794e7ab9EbEfC1e710b4F4EC6F6C1
Native address: oasis1qrjzcve7y6qp3nqs3n7ghavw68vkdh3epcv64ega
```

2. 트랜잭션에서 주소록 사용

주소록에 저장된 이름을 직접 사용하여 토큰 전송:

```
$ oasis account transfer 100 TEST meghan
```

이 명령어는 내부적으로 `meghan` 을 실제 주소로 해결하여 트랜잭션을 실행합니다.

3. 관리 작업

엔트리 이름 변경

```
$ oasis addressbook rename mike mark
```

엔트리 삭제

```
$ oasis addressbook remove mark
```

에러 처리 및 예외 상황

1. 주요 에러 유형

이름 관련 에러

- 중복된 이름 사용
- 잘못된 식별자 형식
- 지갑 계정과 이름 충돌

주소 관련 에러

- 잘못된 주소 형식
- 지원되지 않는 주소 유형
- Ethereum-Native 주소 불일치

시스템 에러

- 설정 파일 저장 실패
- 존재하지 않는 엔트리 접근

2. 에러 처리 전략

사용자 친화적 메시지

```
// 존재하지 않는 엔트리
fmt.Errorf("address named '%s' does not exist in the address book", name)

// 중복된 이름
fmt.Errorf("address named '%s' already exists in the address book", name)

// 잘못된 주소 형식
fmt.Errorf("malformed address '%s': %w", address, err)
```

복구 가능한 에러 처리

- 설정 파일 자동 백업
- 검증 실패 시 상세 정보 제공
- 부분적 복구 시도

보안 고려사항

1. 입력 검증

- 모든 사용자 입력에 대한 엄격한 검증
- 주소 형식 검증을 통한 잘못된 데이터 방지
- 식별자 이름 규칙 강제

2. 데이터 무결성

- 주소 변환 시 일치성 확인
- 설정 파일 저장 전 검증
- 로딩 시 전체 데이터 검증

3. 권한 관리

- 설정 파일 접근 권한 제한
- 민감한 정보 노출 방지

성능 최적화

1. 메모리 효율성

- 지연 로딩을 통한 메모리 사용량 최적화
- 불필요한 객체 생성 방지

2. 검색 성능

- 해시맵을 사용한 O(1) 검색
- 정렬된 출력을 위한 효율적인 정렬 알고리즘

3. 파일 I/O 최적화

- 변경 사항이 있을 때만 설정 파일 저장
- 원자적 파일 쓰기를 통한 데이터 안전성 보장

확장성 및 향후 개선 방안

1. 기능 확장 가능성

- 주소록 엔트리에 메타데이터 추가 (설명, 태그 등)
- 그룹 기능을 통한 주소 분류
- 가져오기/내보내기 기능

2. 통합 개선

- 다른 지갑 시스템과의 연동
- 외부 주소록 서비스 연동
- 클라우드 동기화 기능

3. 사용자 경험 개선

- 자동 완성 기능
- 주소 별칭 시스템
- 시각적 주소 확인 도구

이 주소록 관리 시스템은 사용자의 편의성을 크게 향상시키며, 복잡한 주소 관리를 단순화하여 Oasis CLI의 전반적인 사용성을 개선하는 핵심 기능입니다.